

Architectural Blueprint and Feasibility Analysis for On-Device Geospatial Compilation: The Zero-Cost Edge-Compute Paradigm

Executive Summary

The paradigm of modern mobile application architecture has long been defined by a reliance on cloud-hosted infrastructure for heavy computational tasks. In the domain of geospatial applications, such as offline-first hiking platforms, this traditional model involves server-side clusters executing Extract, Transform, Load (ETL) pipelines. These pipelines ingest massive raw OpenStreetMap (OSM) Protocolbuffer Binary Format (PBF) files and Digital Elevation Models (DEMs), compile them into optimized vector tiles, and serve them to client devices via global Content Delivery Networks (CDNs). While effective, this architecture incurs continuous, scaling cloud compute and storage costs that grow linearly with user acquisition.

The proposed "zero-cost server architecture" represents a radical departure from this model. By shifting the spatial ETL pipeline entirely to the user's mobile device, the cloud footprint is effectively reduced to zero. In this framework, the mobile application directly downloads raw OSM .osm.pbf files and elevation GeoTIFFs from public mirrors (e.g., Geofabrik). The user's device then acts as an autonomous edge-compute node, compiling these raw datasets locally into optimized basemap.pmtiles and terrain.pmtiles archives stored directly on the device's solid-state drive (SSD).

This report delivers an exhaustive technical feasibility analysis and architectural blueprint for engineering this native "On-Device Map Compiler." The research is structured around four critical pillars required to realize this capability: bypassing WebAssembly (WASM) memory limits via native metal execution, engineering a low-memory stream-parsing architecture to avert operating system termination, implementing robust background execution models to allow prolonged compilation, and orchestrating the final tile assembly.

Bypassing WebAssembly Constraints: The Native Metal Bridge

The initial cross-platform hypothesis for on-device compilation often involves compiling a system-level language like Rust to WebAssembly (WASM) and executing it within a Capacitor WebView. While this approach offers seamless JavaScript integration, it possesses catastrophic architectural limitations when processing geospatial data at a planetary or regional scale.

The Memory Bottleneck of WebViews

Modern mobile operating systems impose draconian constraints on WebView execution to

preserve system stability and battery life. On iOS, the WebKit engine operates under the strict oversight of the Jetsam daemon, which aggressively monitors memory consumption. WebViews that exceed 1.5 to 2 GB of RAM are routinely and silently terminated by the operating system¹. A highly compressed 700MB OSM PBF file represents an immensely dense dataset. Because the PBF format relies on delta encoding and string deduplication, parsing the file expands the data payload exponentially in memory³. Reconstructing the topological graph of Nodes, Ways, and Relations for a region of this size requires several gigabytes of volatile memory⁴. Consequently, executing a WASM-compiled Rust binary inside a Capacitor WebView to parse a 700MB PBF is mathematically guaranteed to breach the memory ceiling and trigger an Out-Of-Memory (OOM) termination.

To overcome this fundamental limitation, the compilation engine must entirely bypass the WebView sandbox. The architecture requires executing a natively compiled Rust binary directly on the iOS and Android metal, leveraging the device's maximum hardware capabilities while interfacing with the JavaScript frontend through a specialized bridge.

The Capacitor and UniFFI Architectural Integration

The proposed architecture relies on a tri-layer communication bridge. The core compilation logic is written in pure Rust, providing zero-cost abstractions, memory safety, and aggressive compiler optimizations. This Rust core is compiled natively for the target hardware architectures (aarch64-apple-ios for iOS and aarch64-linux-android for Android).

To connect this low-level binary to the high-level application, the architecture leverages Mozilla's UniFFI (Universal Foreign Function Interface). UniFFI is a multi-language bindings generator that consumes Rust code annotated with specific procedural macros (such as `#[uniffi::export]`) and automatically generates the necessary C-style FFI headers alongside idiomatic Swift and Kotlin wrappers⁵. This automation eliminates the traditionally error-prone process of manually writing JNI (Java Native Interface) boilerplate for Android or Objective-C bridging headers for iOS⁶.

Bridge Component	Technology	Primary Function	Memory Constraint
Frontend UI	React/Vue (JavaScript)	Captures user bounding box and displays progress	Restricted to WebView limits (~1.5 GB)
Native Plugin	Capacitor (Swift/Kotlin)	Intercepts JS calls and routes them to the OS	Governed by Native OS Application limits
FFI Layer	Mozilla UniFFI	Translates data types between Rust	Near-zero overhead, safe

		and Swift/Kotlin	boundary
Compilation Core	Rust Native Binary	Executes stream parsing and tile encoding	Access to maximum device physical RAM

The integration into the mobile ecosystem is managed through a Capacitor Native Plugin. Plugins in Capacitor enable the JavaScript frontend to interface directly with Native APIs⁸. The plugin acts as a modular proxy, exposing a consistent, cross-platform JavaScript API (e.g., `Compiler.startJob()`) while executing drastically different native operations beneath the surface⁸.

iOS Integration via Swift Package Manager

Recent evolutions in the Apple and Capacitor ecosystems necessitate a modernized approach to iOS dependency management. With the release of Capacitor 8, the framework has transitioned from relying on CocoaPods to utilizing the Swift Package Manager (SPM) as a first-class citizen¹¹. The native plugin must be structured to support SPM seamlessly. This involves generating a `Package.swift` file that defines the plugin target. The Swift Package Manager configuration references the UniFFI-generated Swift bindings and links the compiled Rust code, which is packaged as an `.xcframework` to support both physical ARM devices and simulators¹¹. Within the Swift implementation, the plugin class extends the `CAPPlugin` and `CAPBridgedPlugin` protocols, ensuring that the `@objc` decorators expose the functions to Capacitor's dynamic runtime¹⁴.

Android Integration via JNI and Kotlin

On the Android platform, the Rust library is compiled into a shared object (`.so`) using the Android NDK. The UniFFI toolchain generates the corresponding Kotlin classes that map directly to the Rust functions⁶. The Capacitor Android plugin, written in Kotlin, loads the native `.so` library during the `load()` lifecycle method and delegates the heavy computational commands to the Rust runtime.

By utilizing this architecture, the heavy lifting of PBF parsing and MVT encoding occurs entirely outside the JavaScript V8 or JavaScriptCore engines. The application executes at native C/C++ velocities with direct access to the device's CPU caching, multiple cores, and the maximum permitted physical RAM⁶.

Low-Memory Stream Processing: Ingesting OpenStreetMap Data

Processing a 700MB OSM PBF file into renderable vector tiles presents a formidable memory management challenge. The OSM data model is not a standard feature-class hierarchy; rather, it operates as a directed, attributed graph composed of three foundational primitives⁴. Nodes store geographic coordinates (latitude and longitude) and optional tags. Ways are ordered

sequences of node references that form linear features (such as highway=path trails) or closed polygons (such as lakes or buildings). Relations establish higher-order topological groupings, enabling complex multipolygons with inner and outer rings⁴.

Because a Way carries only integer references to Node IDs rather than inline coordinates, geometry reconstruction requires a massive database join⁴. Attempting to load millions of nodes into a standard Rust HashMap to resolve these geometries will instantly exhaust the available RAM on any mobile device.

The Anatomy of the Protocolbuffer Binary Format (PBF)

To avoid OS termination, the architecture must implement a strict stream-parsing methodology. A .osm.pbf file is a continuous, sequential concatenation of length-prefixed blocks designed for high-throughput spatial ETL³. Each block begins with a 4-byte big-endian integer declaring the length of the subsequent BlobHeader. The BlobHeader declares the payload size, which is followed by the Blob itself, containing compressed data (typically zlib or lz4)³.

Once decompressed, the payload reveals either a HeaderBlock or a PrimitiveBlock. A PrimitiveBlock contains the actual OSM entities. To achieve high compression, the format employs two critical techniques. First, string deduplication: each PrimitiveBlock opens with a StringTable, an array of byte strings indexed from 1 that deduplicates every tag key and value used within that block³. Second, delta encoding: object IDs, node references, and coordinates are stored as signed deltas, meaning each value is the difference from its predecessor, zig-zag encoded into a variable-length integer (varint)³.

The Two-Pass Parsing Architecture

A memory-safe Rust compiler utilizes a low-level parsing library, such as `rosm_pbf_reader` or `osmpbf`, to read the file sequentially without loading the entire structure into memory¹⁸. The architecture employs a two-pass algorithm to reconstruct the geometry.

Pass 1: Coordinate Indexing and Semantic Filtering The first pass streams through the PBF, targeting only Node elements. As the parser decodes each PrimitiveBlock, it first analyzes the StringTable. If the StringTable lacks keys relevant to the hiking application—specifically `sac_scale` for trail difficulty, `highway` for paths, `waterway` for streams, or contour-related tags—the entire block can be skipped without incurring the CPU cost of deserializing the individual entities³.

When a relevant block is processed, the parser decodes the ZigZag delta coordinates. To maintain a negligible RAM footprint, these coordinates are not kept in memory. Instead, they are flushed directly to a highly optimized, disk-backed embedded database.

While SQLite is a common choice, its relational overhead and necessity to map Rust types to SQL schemas can introduce latency²⁰. The architecture instead utilizes `redb`, a pure-Rust, embedded key-value store that provides thread-safe, ACID-compliant B-Tree storage directly on the device's SSD²⁰. The system writes the decoded Longitude and Latitude to the `redb` instance, keyed by the 64-bit Node ID, creating a massive but out-of-core spatial index.

Pass 2: Geometry Reconstruction The second pass rewinds the file stream and iterates exclusively through Way and Relation elements. When a relevant Way is identified (e.g., a trail

tagged with `sac_scale=alpine_hiking`), the parser extracts the ordered sequence of Node IDs⁴. For each Node ID, the system queries the local redb database. Because redb utilizes memory-mapped files and efficient B-Tree lookups, the coordinate retrieval is exceedingly fast, reading directly from the SSD cache²⁰. The continuous LineString or Polygon geometry is materialized in memory just long enough to be passed to the transformation and encoding pipeline. Once processed, the geometry is dropped, preventing heap bloat and ensuring the memory usage remains flat regardless of the total file size.

Geospatial Transformation: Projection, Simplification, and Clipping

Once the raw geometry of a trail or waterway is reconstructed in memory, it exists in the WGS 84 geographic coordinate system (EPSG:4326), represented by latitude and longitude degrees¹⁶. However, modern slippy map clients (such as MapLibre GL JS) require data to be presented in planar, Cartesian coordinates utilizing the Web Mercator projection (EPSG:3857)²³.

Mathematical Projection to Web Mercator

The Web Mercator projection is a cylindrical map projection that slightly distorts the standard Mercator by using spherical formulas at all scales, despite the underlying geographic coordinates being defined on the WGS 84 ellipsoidal datum²⁴. This simplifies computational overhead, which is critical for edge computing²⁴.

The Rust pipeline must iterate over every vertex in the reconstructed geometry and apply the transformation mathematically. Utilizing libraries like `toolbox_rs` or custom implementations, the conversion translates geographic degrees into meters from the prime meridian and equator²⁵.

Given a longitude (λ) and geodetic latitude (φ) in radians, and utilizing the equatorial radius of the WGS 84 spheroid ($R = 6,378,137.0$ meters), the formulas are defined as:

$$x = R \cdot \lambda$$

$$y = R \cdot \ln \left(\tan \left(\frac{\pi}{4} + \frac{\varphi}{2} \right) \right)$$

This conversion yields a flat Cartesian plane where the map is defined as a perfect square, cutting off polar coverage at approximately 85.051129° North and South²⁴.

Vector Simplification via Ramer-Douglas-Peucker

High-resolution GPS traces and detailed OSM ways contain a density of vertices that exceeds the rendering capabilities of a mobile screen, leading to inflated tile sizes and degraded rendering performance²⁷. To generate an optimized basemap.pmtiles archive, the projected

geometries must be mathematically simplified based on the target zoom level.

The architecture utilizes the Ramer-Douglas-Peucker (RDP) algorithm, provided by the Rust geo or simplification crates²⁹. The RDP algorithm is a line simplification technique that reduces the number of points defining a shape. It operates by drawing a line segment between the first and last points of a curve, then finding the point furthest from this segment. If this maximum

distance is less than a predefined error threshold (ϵ), the intermediate points are discarded. If the distance exceeds ϵ , the algorithm recursively splits the curve at that point²⁷.

The ϵ threshold is dynamically scaled according to the zoom level being compiled. For a global overview at zoom level 5, an aggressive ϵ value strips away microscopic bends in a trail, maintaining only the macroscopic shape. For zoom level 14, ϵ is minimized to retain the granular switchbacks necessary for hiking navigation³¹.

Tile Boundary Clipping via Sutherland-Hodgman

As geometries are translated into the tile grid system, long trails or large water bodies inevitably span multiple tile boundaries. A vector tile must only contain the geometry strictly within its bounding box (plus a minor rendering buffer to prevent visual artifacts on tile edges)³².

To enforce this, the pipeline applies the Sutherland-Hodgman polygon clipping algorithm. This algorithm finds the intersection between an arbitrary subject polygon (or linestring) and a convex clip polygon (the rectangular tile boundary)³⁴. The Rust implementation, utilizing libraries like wbtology or clipper2, iteratively processes the geometry against the four clipping planes of the tile (left, right, bottom, top)³⁶.

If a trail segment crosses the tile boundary, the algorithm calculates the exact intersection point using parametric line equations, inserts a new vertex at the boundary, and discards the segment falling outside the tile³⁴. This ensures that when the tile is finally encoded, no extraneous geographic data inflates the byte size³³.

Terrain and Topography: Processing Elevation GeoTIFFs

A fundamental requirement for an offline hiking application is the visualization of topography. Topographical data is distributed as Digital Elevation Models (DEMs) packaged in GeoTIFF format³⁹. A GeoTIFF is a Tagged Image File Format that embeds spatial reference information (GeoKeys) and stores scalar data—in this case, elevation in meters above sea level—rather than RGB color values⁴¹.

Asynchronous Windowed Reading

Raw DEM files covering mountainous regions can span multiple gigabytes. The architecture must avoid utilizing heavyweight, C-based libraries like GDAL, which are notoriously difficult to cross-compile for mobile targets and introduce significant overhead⁴³. Instead, the pipeline

relies on pure-Rust libraries such as `geotiff-reader` or `async-geotiff`⁴³.

These libraries permit asynchronous, windowed reading of the raster data. The parser first reads the TIFF headers and Image File Directories (IFDs) to establish the affine transformation matrix, the Coordinate Reference System (CRS), and the internal block layout⁴². Rather than loading the entire matrix, the system requests a specific spatial window corresponding to the geographic bounding box of the tile currently being processed⁴³. The Rust core fetches only the required chunks of the u16 or f32 scalar elevation arrays into memory⁴⁸.

Contour Generation via Marching Squares

To render elevation visually, the application must extract vector contour lines from the scalar raster grid. This is achieved using the Marching Squares algorithm, facilitated by crates such as `contour` or `contour-isobands`⁵⁰.

The user's configuration defines the desired elevation intervals (e.g., minor contours every 10 meters, major contours every 50 meters)³⁹. The Marching Squares algorithm treats the elevation

array as a 2D scalar field, $f(x, y)$. It analyzes the field by dividing it into a grid of squares defined by four adjacent pixel values⁵³. By evaluating the scalar values at the corners against the target threshold (the contour level), the algorithm generates a binary state for each corner (above or below the threshold)⁵³.

Using linear interpolation along the edges of the square, the algorithm identifies the precise sub-pixel coordinates where the contour crosses the boundary. These segments are connected across the grid to produce continuous, closed `MultiLineString` isolines representing the topographic contours⁵¹.

Once generated, these contour lines are subjected to the exact same transformation pipeline as the OSM data: projection to Web Mercator, RDP simplification to remove jagged pixelation artifacts, and Sutherland-Hodgman clipping to the tile boundaries³¹.

MVT Serialization and PMTiles Assembly

With the geographic features—trails, waterways, and contours—fully transformed, simplified, and clipped, they must be packaged for the rendering engine. The standard protocol for this is the Mapbox Vector Tile (MVT) format.

MVT utilizes Google Protocol Buffers to serialize geometric data. Rather than storing global Web Mercator coordinates, MVT translates the data into a local, integer-based coordinate space relative to the tile itself³². The standard extent of an MVT tile is a 4096×4096 grid³³. By snapping vertices to this integer grid and applying a "Command Integer" structure (`MoveTo`, `LineTo`, `ClosePath`), the geometry is heavily compressed⁵⁷. Rust libraries like `fast-mvt` perform this conversion using ZigZag encoding to minimize the variable-length integers, ensuring the tile size remains diminutive³⁸.

Streaming to the PMTiles Archive

The final output is not a directory of millions of individual `.pbf` files, which would devastate the mobile file system's inode table. Instead, the application generates a PMTiles archive (v3

specification)⁵⁸. PMTiles is a single-file, cloud-native archive format uniquely designed for Z/X/Y addressed tiled data⁵⁹.

The Rust pipeline integrates libraries such as pmtiles2 or gpq_tiles_core to construct the archive⁶⁰. The architecture streams the compressed MVT byte arrays directly to a disk-backed std::fs::File writer⁶². The PMTiles writer handles the complex spatial indexing required by the format, utilizing space-filling curves (such as Hilbert or Morton ordering) to ensure that geographically adjacent tiles are stored consecutively on disk, optimizing future read latency³².

Assembly Stage	Data Structure	Memory Strategy	Storage Destination
Geometry Clipping	geo_types::MultiLineString	Volatile Heap (Dropped immediately)	None
MVT Serialization	Protobuf Byte Array	Volatile Heap (Transferred to compressor)	None
Tile Compression	Gzip / Brotli / Zstd	Streamed via async-compression	None
Archive Assembly	PMTiles Directory & Data	Buffered write via pmtiles2::to_writer	Local SSD (basemap.pmtiles)

Furthermore, the writer automatically calculates cryptographic hashes of each tile payload. If the hiking region contains vast expanses of identical terrain (e.g., empty ocean or flat plains), the deduplication engine ensures the byte array is written only once, with multiple directory entries pointing to the same offset, radically shrinking the final archive size⁶⁰.

Overcoming Mobile Operating System Background Limits

Compiling regional maps from raw PBFs and GeoTIFFs represents a severe computational workload. Processing a densely mapped area can require 5 to 15 minutes of sustained CPU activity⁶³. Expecting a user to keep the application active in the foreground, with the screen illuminated, for this duration is an unacceptable user experience. The process must execute asynchronously in the background. However, modern mobile operating systems actively combat background processes to preserve battery endurance and thermal stability².

Android: WorkManager and Foreground Services

The Android operating system categorizes applications into distinct power buckets (active, working set, frequent, restricted) based on user interaction, throttling network and CPU access accordingly². For tasks that are deferrable, Google mandates the use of WorkManager, which schedules execution during optimal system states⁶⁴.

However, map compilation is immediate, user-initiated, and exceeds the maximum execution windows permitted for standard background services⁶⁶. To bypass these restrictions, the architecture must elevate the task to a Foreground Service via the WorkManager API⁶⁴. By invoking `SystemForegroundService` and declaring a `foregroundServiceType` of `specialUse` or `dataSync` in the `AndroidManifest.xml`, the application signals to the OS that it is performing a critical, user-aware operation⁶⁸.

A Foreground Service requires the application to display a persistent, non-dismissible notification⁶⁴. This notification grants the Rust process immunity from the aggressive Doze modes, allowing it to hold a CPU wake-lock and execute the tile compilation uninterrupted until completion⁶⁶.

iOS: BGTaskScheduler and Proprietary Heuristics

Apple's iOS environment utilizes a profoundly restrictive background execution model. Under normal circumstances, applications are suspended by the OS within seconds of being minimized, preventing any further CPU cycles².

To process data in the background, the application must hook into the BGTaskScheduler framework, utilizing a BGProcessingTask⁶⁹. Unlike Android's explicit Foreground Service, iOS schedules BGProcessingTask execution based on proprietary heuristics, factoring in the user's charging habits, the device's thermal state, and historical app usage⁶⁹. Consequently, execution is deferred and cannot be guaranteed to run immediately upon minimization⁷⁰.

To operate within this constraint, the application UI must clearly inform the user that the map generation has been queued and will occur silently while the device is charging. Alternatively, the application can leverage audio or location-tracking background modes if the user begins their hike immediately, though utilizing these APIs strictly to keep a compilation thread alive violates Apple's App Store review guidelines and risks rejection⁷⁰. It is noteworthy that regulatory pressure, such as the European Union's Digital Markets Act (DMA), is forcing Apple to introduce new interoperability APIs by 2026, which may provide more explicit background execution permissions for complex companion applications in the near future⁷¹.

Thermal Throttling and Concurrency Governance

The actual computation is orchestrated by Rust's rayon crate, which provides a work-stealing thread pool to parallelize the decoding of PBF blocks and the heavy mathematics of MVT serialization⁷². By default, rayon will spawn threads equal to the number of logical cores available⁷⁴.

On mobile Systems-on-a-Chip (SoCs), which feature asymmetric core layouts (e.g., ARM big.LITTLE architectures with Performance and Efficiency cores), saturating all threads will rapidly induce thermal throttling. The device temperature will spike, prompting the OS to slash CPU frequencies, ultimately slowing down the compilation and risking a thermal shutdown⁷².

To mitigate this, the architecture explicitly configures the `ThreadPoolBuilder`. By setting `max_threads` to a conservative value (e.g., utilizing only the Performance cores or leaving 2-3 cores entirely free for the OS and UI thread), the compilation proceeds at a sustained, thermally stable pace without degrading the overall user experience of the device⁷².

Bridging Asynchronous Progress Reporting

While the background compilation proceeds, the application must provide dynamic feedback to the user interface. This is achieved through UniFFI's robust asynchronous capabilities, specifically utilizing the `callback_interface` attribute⁷⁵.

The architecture defines a Rust trait annotated with `#[uniffi::export(callback_interface)]`, outlining methods such as `on_progress(percentage: f32, status: String)`⁷⁷. During initialization, the Swift or Kotlin native layer instantiates an object conforming to this interface and passes it across the FFI boundary into the Rust engine⁷⁷.

As the rayon workers iterate through PBF blocks and generate GeoTIFF contours, they periodically invoke this callback⁷⁸. The callback bridges back into the native mobile layer, which then emits an event through Capacitor's standard listener architecture (e.g., `MyPlugin.addListener('compilationProgress', ...)`). This continuous stream of data allows the JavaScript React or Vue frontend to render a smooth progress bar and status text¹⁵. Crucially, if the app is minimized and restored, the persistent Rust background thread simply continues firing the callback, allowing the UI to instantly reflect the current state without querying an external server.

Concrete Architectural Blueprint

Synthesizing these pillars yields a deterministic data-flow blueprint for the zero-cost edge-compute compiler:

1. **Ingestion Request (Capacitor JS):** The user defines a bounding box for their target hiking region. The JavaScript UI triggers the Capacitor Native Plugin, passing the bounding box coordinates and selected map features.
2. **Data Fetching (Native OS / Rust):** The device initiates a direct HTTP download of the regional `.osm.pbf` from an external mirror and the corresponding GeoTIFF DEMs. The files are streamed directly to the local application storage sandbox, bypassing cloud intermediaries.
3. **Execution Elevation (OS Layer):** The Native Plugin triggers a Foreground Service (Android) or schedules a `BGProcessingTask` (iOS), ensuring the OS scheduler permits prolonged CPU cycles and prevents premature termination.
4. **Rust Initialization (UniFFI):** The Native layer invokes the Rust engine via UniFFI, passing the file paths, the user's bounding box, and the callback interface pointer for progress reporting.
5. **Pass 1 - Coordinate Extraction (Rust):** The `rosm_pbf_reader` parses the PBF. Nodes matching hiking infrastructure are indexed. Their projected Web Mercator coordinates are extracted, and the data is written directly to an embedded `redb` database.
6. **Pass 2 - Feature Assembly & Vectorization (Rust):**

- The stream parser rewinds and extracts Ways and Relations. Node coordinates are retrieved from the SSD-backed redb.
 - Geometries are simplified via Ramer-Douglas-Peucker according to the target zoom levels and clipped via Sutherland-Hodgman per tile boundary.
 - Features are encoded into MVT protobufs using ZigZag encoding.
 - The pmtiles2 crate writes the serialized MVTs sequentially into basemap.pmtiles, applying deduplication and Hilbert curve sorting.
7. **Terrain Assembly (Rust):**
 - The geotiff-reader fetches asynchronous data windows from the elevation raster.
 - The Marching Squares algorithm generates isolines based on user-defined intervals.
 - Contour vectors are simplified, clipped to the tile, and MVT-encoded.
 - The serialized tiles are written to terrain.pmtiles.
 8. **Finalization & Notification:** Temporary redb files are purged from the storage. The UniFFI callback signals 100% completion. The OS drops the foreground service notification or concludes the background task.
 9. **Offline Rendering:** The Capacitor WebView initializes MapLibre GL JS combined with the pmtiles protocol, rendering the maps instantly from the local SSD archives, requiring zero network connectivity for the duration of the hike.

This blueprint demonstrates that by capitalizing on the vast compute power of modern mobile architectures, the "FreeHike" application can effectively shift server compute costs entirely to the consumer's idle hardware. By enforcing strict memory governance, leveraging pure-Rust spatial libraries, and navigating operating system background constraints, this zero-cost architecture provides a resilient, highly scalable foundation for next-generation geospatial software.

Works cited

1. Mobile: A different way using Rust? - codethoughts, <https://codethoughts.io/posts/2024-06-25-mobile-a-different-way/>
2. App Background Activity 2026: OS Restrictions Guide, <https://alexrooter.com/os-background-limits/>
3. PBF File Structure Deep Dive - OSM Data Processing & QA Pipelines, <https://www.osm-data-processing.org/osm-data-fundamentals-architecture/pbf-file-structure-deep-dive/>
4. OSM Data Fundamentals & Architecture, <https://www.osm-data-processing.org/osm-data-fundamentals-architecture/>
5. mozilla/uniffi-rs: a multi-language bindings generator for rust - GitHub, <https://github.com/mozilla/uniffi-rs>
6. Diamond in the Rust: Sharing code across mobile platforms - LINEヤフー Tech Blog, <https://techblog.lycorp.co.jp/en/20241002a>
7. Crossplatform Business Logic in Rust - ForgeStream - IDVerse®, <https://forgestream.idverse.com/blog/20251105-crossplatform-business-logic-in-rust/>
8. Capacitor Plugins | Capacitor Documentation, <https://capacitorjs.com/docs/plugins>
9. Using Capacitor Plugins in Ionic: Best Practices for Native Features - Djamware,

- <https://www.djamware.com/post/using-capacitor-plugins-in-ionic-best-practices-for-native-features>
10. Qwik Hybrid Native App Overview | Guides Qwik Documentation, <https://qwik.dev/docs/guides/capacitor/>
 11. Swift Package Manager | Capacitor Documentation, <https://capacitorjs.com/docs/ios/spm>
 12. Swift Package Manager and Capacitor - Ionic Blog, <https://ionic.io/blog/swift-package-manager-and-capacitor>
 13. iOS native implementation - ODC Documentation, https://success.outsystems.com/documentation/outsystems_developer_cloud/building_apps/mobile_apps/use_mobile_plugins/building_a_capacitor_plugin_from_scratch/ios_native_implementation/
 14. Custom Native iOS Code | Capacitor Documentation, <https://capacitorjs.com/docs/ios/custom-code>
 15. Capacitor iOS Plugin Guide, <https://capacitorjs.com/docs/plugins/ios>
 16. Node - OpenStreetMap Wiki, <https://wiki.openstreetmap.org/wiki/Node>
 17. GitHub - folknor/pbfhogg: Rust library and CLI for reading, writing, and merging OpenStreetMap PBF with parallel decoding and write support, <https://github.com/folknor/pbfhogg>
 18. rosm_pbf_reader - Rust - Docs.rs, https://docs.rs/rosm_pbf_reader
 19. rosm_pbf_reader - crates.io: Rust Package Registry, https://crates.io/crates/rosm_pbf_reader
 20. Introducing Native DB: A fast, multi-platform embedded database for Rust - Reddit, https://www.reddit.com/r/rust/comments/18lxb2n/introducing_native_db_a_fast_multiplatform/
 21. vincent-herlemont/native_db: Drop-in embedded database in Rust - GitHub, https://github.com/vincent-herlemont/native_db
 22. tauri-plugin-reddb-cache - crates.io: Rust Package Registry, <https://crates.io/crates/tauri-plugin-reddb-cache>
 23. Mercator—ArcGIS Pro | Documentation, <https://pro.arcgis.com/en/pro-app/3.6/help/mapping/properties/mercator.htm>
 24. Web Mercator projection - Wikipedia, https://en.wikipedia.org/wiki/Web_Mercator_projection
 25. toolbox_rs::mercator - Rust - Docs.rs, https://docs.rs/toolbox-rs/latest/toolbox_rs/mercator/index.html
 26. Mercator - OpenStreetMap Wiki, <https://wiki.openstreetmap.org/wiki/Mercator>
 27. Ramer-Douglas-Peucker line simplification - Rosetta Code, https://rosettacode.org/wiki/Ramer-Douglas-Peucker_line_simplification
 28. Generate Contours Command - CivilGEO Knowledge Base, <https://knowledge.civilgeo.com/generate-contours-command/>
 29. simplification - PyPI, <https://pypi.org/project/simplification/>
 30. geo - Rust - Docs.rs, <https://docs.rs/geo/>
 31. MicroJSON tiling, <https://bengtl.github.io/microjson/tiling/>
 32. Architecture - Martin Tile Server Documentation - MapLibre,

- <https://maplibre.org/martin/architecture/>
33. Functions for Encoding Mapbox Vector Tiles | ClickHouse Docs, <https://clickhouse.com/docs/sql-reference/functions/geo/mvt>
 34. Sutherland-Hodgman polygon clipping - Rosetta Code, https://rosettacode.org/wiki/Sutherland-Hodgman_polygon_clipping
 35. Using Sutherland-Hodgman Algorithm to customize collision masking on Unity 2D - Medium, <https://medium.com/@vinchenamigo/using-sutherland-hodgman-algorithm-to-customize-collision-masking-on-unity-2d-9095e1ab6132>
 36. Boolean operations on polygons - Wikipedia, https://en.wikipedia.org/wiki/Boolean_operations_on_polygons
 37. arclabs561/subsume: Region embeddings for entailment and set containment - GitHub, <https://github.com/arclabs561/subsume>
 38. nyurik/fast-mvt: Fast Mapbox Vector Tile (MVT) reader and writer - GitHub, <https://github.com/nyurik/fast-mvt>
 39. gdal_contour — GDAL documentation, https://gdal.org/en/stable/programs/gdal_contour.html
 40. Use elevation | Guides - MapTiler documentation, <https://docs.maptiler.com/guides/location-services/elevation/>
 41. Dem data - geotifs | Forums - SideFX, <https://www.sidefx.com/forum/topic/28206/>
 42. Understanding GeoTIFFs and Rendering Them in Leaflet - The Miners - Codeminer42, <https://blog.codeminer42.com/understanding-geotiffs-and-rendering-them-in-leaflet/>
 43. Introducing Async-GeoTIFF - Development Seed, <https://developmentseed.org/async-geotiff/latest/blog/2026/02/03/introducing-async-geotiff/>
 44. Accelerating GeoTIFF readers with Rust :: FOSS4G 2025 - pretalx, <https://talks.osgeo.org/foss4g-2025/talk/MRPVGL/>
 45. geotiff-reader - crates.io: Rust Package Registry, <https://crates.io/crates/geotiff-reader>
 46. GeoTransform in geotiff_reader::transform - Rust - Docs.rs, https://docs.rs/geotiff-reader/latest/geotiff_reader/transform/struct.GeoTransform.html
 47. geo_rasterize - Rust - Docs.rs, <https://docs.rs/geo-rasterize>
 48. georaster - crates.io: Rust Package Registry, <https://crates.io/crates/georaster>
 49. georaster - Rust library for accessing geospatial raster images - GitHub, <https://github.com/pka/georaster>
 50. marching-squares - Keywords - crates.io: Rust Package Registry, <https://crates.io/keywords/marching-squares>
 51. contour_isobands - Rust - Docs.rs, <https://docs.rs/contour-isobands>
 52. Generate Contour Data from Raster - FME Support Center - Safe Software, <https://support.safe.com/hc/en-us/articles/25407759575821-Generate-Contour-Data-from-Raster>
 53. marching_squares_wgpu - Rust - Docs.rs,

- https://docs.rs/marching_squares_wgpu
54. Frame of Reference and Roaring Bitmaps | Elastic Blog,
<https://www.elastic.co/blog/frame-of-reference-and-roaring-bitmaps>
 55. GPKG -- GeoPackage raster — GDAL documentation,
<https://gdal.org/en/stable/drivers/raster/gpkg.html>
 56. ST_AsMVTGeom - PostGIS,
https://postgis.net/docs/manual-3.7/it/ST_AsMVTGeom.html
 57. mvt - Rust - Docs.rs, <https://docs.rs/mvt>
 58. GitHub - protomaps/PMTiles: Pyramids of map tiles in a single file on static storage, <https://github.com/protomaps/pmtiles>
 59. PMTiles Concepts | Protomaps Docs, <https://docs.protomaps.com/pmtiles/>
 60. PMTiles in pmtiles2 - Rust - Docs.rs,
<https://docs.rs/pmtiles2/latest/pmtiles2/struct.PMTiles.html>
 61. gpq_tiles_core - Rust - Docs.rs, <https://docs.rs/gpq-tiles-core>
 62. pmtiles2 - Rust - Docs.rs, <https://docs.rs/pmtiles2>
 63. fast_osmpbf - Rust - Docs.rs, <https://docs.rs/fast-osmpbf>
 64. Services overview | Background work - Android Developers,
<https://developer.android.com/develop/background-work/services>
 65. WorkManager: Back to the Foreground - Persistent Background Tasks in Android,
<https://www.classcentral.com/course/youtube-workmanager-back-to-the-foreground-211645>
 66. Demystifying Android Services: What, Why & How | by Dilip Patel | Medium,
<https://dilipp817.medium.com/demystifying-android-services-what-why-how-964c87f8da29>
 67. Foreground service vs WorkManager for location tracking - Stack Overflow,
<https://stackoverflow.com/questions/64219756/foreground-service-vs-workmanager-for-location-tracking>
 68. Android WorkManager without its foreground service - Stack Overflow,
<https://stackoverflow.com/questions/78779719/android-workmanager-without-its-foreground-service>
 69. [Part 2] Background work in iOS. In Part 1, we explored the... | by Sagar Pant | Medium,
<https://medium.com/@sagarpant/part-2-background-work-in-ios-108a1d29bae0>
 70. Background tasks and limits on iOS/Android — Ops Runbook — Practical Guide (May 4, 2026) - Sachith Dassanayake,
<https://www.sachith.co.uk/background-tasks-and-limits-on-ios-android-ops-runbook-practical-guide-may-4-2026/>
 71. Interoperability - Digital Markets Act (DMA) - European Commission,
https://digital-markets-act.ec.europa.eu/developer-portal/interoperability_en
 72. How can make this more performant? : r/rust - Reddit,
https://www.reddit.com/r/rust/comments/1dr7rcm/how_can_make_this_more_performant/
 73. Rayon implementation of Triple modular redundancy - help - Rust Users Forum,
<https://users.rust-lang.org/t/rayon-implementation-of-triple-modular-redundancy/89295>

74. Disable parallelism on installation time - Stack Overflow,
<https://stackoverflow.com/questions/78751811/disable-parallelism-on-installation-time>
75. lib.rs - source,
https://mozilla.github.io/uniffi-rs/0.27/internals/api/src/uniffi_bindgen/lib.rs.html
76. Procedural Macros: Attributes and Derives - The UniFFI user guide,
https://mozilla.github.io/uniffi-rs/proc_macro/index.html
77. cosmic-ext-connect-android/docs/guides/PLUGIN_API.md at master - GitHub,
https://github.com/olafkfreund/cosmic-ext-connect-android/blob/master/docs/guides/PLUGIN_API.md
78. Implement callback interfaces for Swift #353 - mozilla/uniffi-rs - GitHub,
<https://github.com/mozilla/uniffi-rs/issues/353>